

Enterprise Doc AI

A Local-First Document Question Answering System

Project Summary

*What we built, the problems we hit,
and how we solved them*

Shahab Azmi · Chitransh Kumar Gupta

A private, on-device assistant for any document — built from scratch.

May 2026

1 Why We Built This

Public AI assistants and hosted RAG services have three real problems for anyone working with private documents:

1 **Privacy.** Your file leaves your computer — the provider receives, stores, and may train on it. Unacceptable for confidential material.

*the document
never stays
on your laptop*

2 **Hallucination.** Small LLMs add hedge phrases (“could lead to”, “analysts believe”) not in the source — and you can't see which sentence grounded the answer.

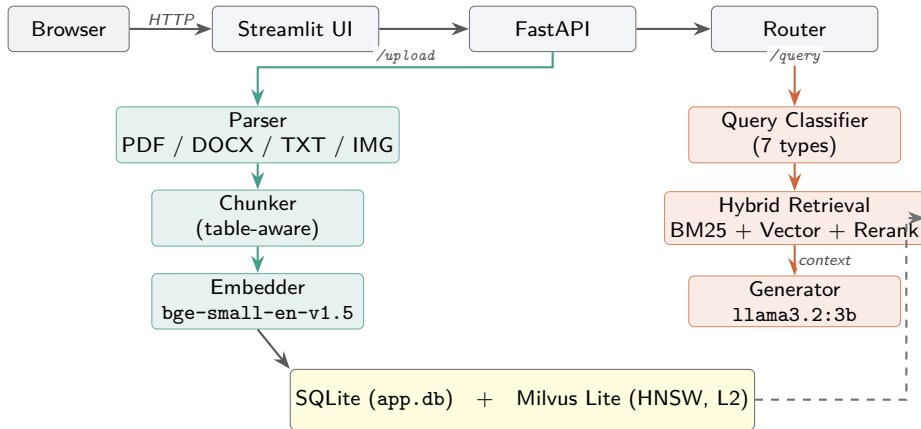
*you can't verify
where the answer
came from*

3 **Cost.** Hosted RAG charges per token, per page, per query. Indexing 1,000 pages and asking 50 follow-up questions adds up fast — you can't experiment freely.

*every page +
every question
= money*

Our goal: a complete document QA system where every piece runs on the user's laptop — on the user's data, with zero cost per query.

2 System Architecture



Solid arrows = write; dashed = read. ■ **Ingest** pipeline. ■ **Query** pipeline. **SQLite is the single source of truth** – vector store rebuilds on startup if missing.

3 How an Uploaded PDF Becomes Searchable

Walk-through. The user drags `NASDAQ_TSLA_2024.pdf` (Tesla 10-K, 1.7 MB) into the chat. Five things happen, in order:

1

Check the file. Look at the file type (PDF/DOCX/TXT/image) and SHA-256-hash the bytes. If this exact file was uploaded before, reuse it and skip the rest.

Tesla 10-K
→ new, continue

2

Read the pages. PyMuPDF walks every page and pulls the text out. Tables are extracted *separately* so revenue rows stay together.

~1,400 raw
text + table blocks

3

Slice the text into small chunks. A 200-page PDF is too big to search at once. Split into windows of **500 chars** with **100 overlap** — one paragraph per chunk.

→ 1,180
searchable chunks

4

Convert each chunk into numbers. The model `bge-small-en-v1.5` turns each chunk into **384 numbers** that capture its meaning. “CEO” and “Chief Executive” end up nearby.

$1,180 \times 384$
numbers

5

Save everything locally. Text → SQLite (source of truth). Numbers → Milvus Lite (fast similarity search). The file is now ready to answer questions.

ready in
≈ 40 s

4 How a Question Becomes a Grounded Answer

Walk-through. The user types “Who is the CEO of NVIDIA?”. Five things happen, in order:

1 **Decide what kind of message this is.** Is it a doc question, chat (“hello”), or live data (“weather”) ? “Who is the...” + PDF loaded $\Rightarrow$ *document question*. *not chat,
not weather*

2 **Decide what KIND of question.** Seven types: extractive, numerical, provenance, list, reasoning, summary, factual. “CEO” is a short fact $\Rightarrow$ **FACTUAL**. *type = Factual
(k = 3)*

3 **Search the chunks — two ways at once.** **Keyword (BM25)** finds chunks with literal “CEO”. **Meaning (Milvus)** finds chunks about *leadership*. Both lists are merged. *~50
candidates*

4 **Pick the THREE sharpest passages.** A *cross-encoder* reads each candidate *with* the question and scores them tightly. Slower, but much sharper than the first two searches. *top 3 chunks*

5 **Generate the answer — strictly.** The local LLM (llama3.2:3b) sees *only* those 3 chunks with a 10-rule prompt and $T = 0$. No outside knowledge. *T = 0,
strict prompt*

Result: “Mr. Huang serves as our President and CEO.” + 3 source citations

5 What Broke When I Tested — and How I Fixed It

Three real bugs, each tied to one of the test PDFs, diagnosed in code and verified after the fix.

1

Tesla 10-K — revenue tables collapsed into one line. *Test:* “Tesla revenue 2024?” *Bug:* Rows like 21,344 23,210 17,933 25,500 merged into one string. *Fix* (`chunker.py`): keep line breaks if ≥ 3 short numeric lines.

After:
\$77,070 M

2

NVIDIA — the LLM invented hedge phrases. *Test:* “Who is the CEO of NVIDIA?” *Bug:* Model added “could lead to”, opened with “Based on the doc...”. *Fix* (`generator.py`): strict prompt + ban-list + regex opener cleaner.

After:
**“Mr. Huang
... President
and CEO.”**

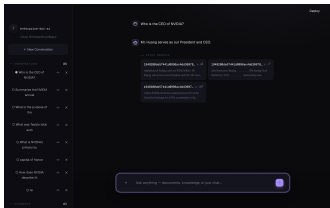
3

NCIC PIA — model paraphrased a verbatim quote. *Test:* “Show the exact paragraph about NCIC.” *Bug:* Model re-worded the paragraph. *Fix* (`query_classifier.py`): EXTRACTIVE+PROVENANCE types bypass the LLM, return raw chunk.

After:
 **≈ 200 ms
100% faithful**

#4 Bi-encoder misses — fixed with parallel BM25 + vector + page-1/acronym/@ bonuses, then cross-encoder rerank. **#5 State loss** — catch Milvus drops, reset & retry; cap LLM context at 3,000 chars; auto-rebuild from SQLite on startup.

6 Live Demo on Three Real PDFs



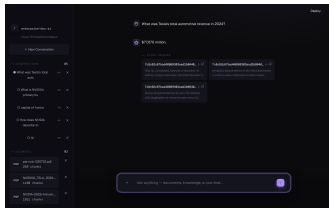
NVIDIA Annual Report

(1,631 chunks)

FACTUAL query.

"Who is the CEO of NVIDIA?"

⇒ *"Mr. Huang serves as our President and CEO."*



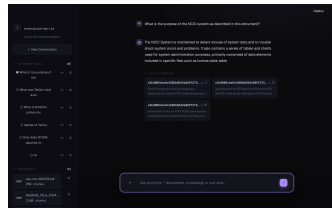
Tesla 10-K (FY24)

(1,180 chunks)

NUMERICAL query.

"Tesla's total automotive revenue in 2024?"

⇒ *"\$77,070 million."*



FBI NCIC PIA

(259 chunks)

REASONING query.

"Purpose of the NCIC system?"

⇒ *"Maintained to detect misuse... troubleshoot system errors..."*

Every answer comes with **three source citations** (file / page / snippet) rendered from chunk metadata, so the user can verify the fact at its source. All three PDFs are indexed at once in one Milvus Lite file (3,070 chunks total) on an Apple M2 with 8 GB RAM.

7 The Whole Project, From Start to End

1. The Problem. Private files need a smart assistant, but hosted services leak the file, hallucinate, and cost money per query.

2. What I Built. A FastAPI + Streamlit chat app where every piece — parser, BM25, embeddings, vector store, LLM, conversations — runs on the user's laptop, offline.

3. How I Tested It. On three diverse PDFs to stress different parts of the pipeline:

Document	Chunks
NVIDIA 2025 Annual Report (dense prose)	1,631
Tesla 10-K FY24 (financial tables)	1,180
FBI NCIC PIA (government reasoning)	259
Total indexed at once	3,070

(The system supports any user-supplied PDF, DOCX, TXT or image — the three above are demonstration cases.)

4. What Broke and How I Fixed It.

- Retrieval misses $\Rightarrow$ Hybrid BM25 + Vector + bonuses.
- Hedge-phrase hallucination $\Rightarrow$ strict prompt + regex.
- Paraphrased quotes $\Rightarrow$ LLM-bypass for 2 query types.
- Tables $\rightarrow$ gibberish $\Rightarrow$ numeric-line heuristic.
- State loss $\Rightarrow$ SQLite-as-truth + auto-rebuild.

5. What Works Now.

NVIDIA "CEO?" $\rightarrow$ **Mr. Huang** (4 words).

Tesla "Revenue 2024?" $\rightarrow$ **\$77,070 M.**

NCIC "Purpose?" $\rightarrow$ multi-sentence reasoning.

All with **3 source citations**, 5–12s, on 8 GB RAM, *no GPU, no cloud*.

6. What's Next. Stronger reranker (bge-reranker-base); SSE token streaming; learned router; multilingual via bge-m3.

Why it matters. A complete RAG pipeline can run *on your laptop, on your data*, without sending a single byte to the cloud.